

Table des Matieres

1. Mecanismes Avances : Ce que le premier document ne disait pas	3
2. Mixture-of-Agents (MoA) : Le mecanisme cle	3
2.1 Architecture MoA a 3 Couches	3
2.2 Impact sur les Agents Cibles	4
3. DAG-Based Code Generation : Le coeur du moteur	4
3.1 Construction du DAG	5
3.2 Contexte Lineaire (Linear Context Management)	5
4. 3-Level Auto-Fix Engine : Correction automatique	5
4.1 Niveau 1 : Corrections par Pattern	6
4.2 Niveau 2 : Corrections par AST	6
4.3 Niveau 3 : Corrections par LLM	6
5. Reflection Multi-Criteres : La cle des 92%	7
6. Routeur RL-Optimise : Apprentissage du routing	7
7. Architecture Unifiee : Integration complete	8
7.1 Agent Developpeur : Architecture Unifiee	8
7.2 Agent SLIDES : Architecture Unifiee	9
8. Infrastructure Critique et Deploiement	9
8.1 Sandbox E2B / Docker	9

8.2 Stack de Deploiement	10
8.3 Schema de Base de Donnees Critique	10
9. Optimisation des Couts : Modele economique	10
10. Roadmap Unifiee : 16 Semaines	11
11. Les 10 Nouveaux Commandements (Addendum)	12

1. Mechanismes Avances : Ce que le premier document ne disait pas

Le premier document (Compilateur d'Architecture Intelligente) fournissait les fondements theoriques et les patterns generaux. Le second document (Guide de Clonage GenSpark) revele les mecanismes operationnels specifiques qui font la difference entre un prototype academique et un systeme de production atteignant 87.8% de taux de succes sur le benchmark GAIA. Cette synthese identifie les sept mecanismes cles qui n'etaient pas couverts ou insuffisamment detailles dans le premier document, et les integre dans une architecture unifiee pour l'Agent Developpeur et l'Agent SLIDES.

Mecanisme	Document 1	Document 2 (GenSpark)	Impact
Mixture-of-Agents (MoA)	Pattern Pipeline/Debat mentionne	9 LLMs en parallele avec voting	CRITIQUE
DAG-based Generation	Non couvert	Generation parallele par niveaux de dependance	CRITIQUE
3-Level Auto-Fix	Correction mentionnee	Pattern > AST > LLM (3 niveaux)	ESSENTIEL
Reflection Multi-Criteres	Pattern Debat	6 criteres ponderes + score confiance	ESSENTIEL
RL-Optimized Router	Routeur basique	Apprentissage par renforcement	AVANCE
Linear Context Management	Non couvert	Evitement des conflits de contexte	IMPORTANT
Template Learning	Auto-amelioration generale	Amelioration continue des patterns	AVANCE

Tableau 1 : Analyse des lacunes entre les deux documents sources

2. Mixture-of-Agents (MoA) : Le mecanisme cle

Le Mixture-of-Agents est le mecanisme fondamental qui distingue un agent mono-modele d'un systeme de production. Plutot que d'utiliser un seul LLM, le MoA orchestre 9 modeles en parallele, chacun avec ses forces specifiques, puis agregge les resultats via un mecanisme de vote multi-criteres. Ce mecanisme est responsable du taux de succes de 87.8% sur GAIA, car il compense les faiblesses de chaque modele par les forces des autres.

2.1 Architecture MoA a 3 Couches

L'architecture MoA se decompose en trois couches distinctes. La premiere couche est le Routeur de Modeles (Model Router) qui analyse chaque sous-tache et selectionne les 2-3 modeles optimaux en fonction du type de tache, de la complexite et du budget. La deuxieme couche est l'Execution Parallele qui lance les modeles selectionnes simultanement, chaque modele produisant sa propre version du resultat. La troisieme couche est l'Agent de Reflexion qui evalue les sorties de chaque modele selon six criteres ponderes, synthee les meilleurs elements et produit un resultat final avec un score de confiance. Si le score de confiance est inferieur a 0.95, le systeme declenche un cycle de raffinement.

Type de Sous-Tache	Modeles Primaires	Modele de Secours	Cout estimatif
Planning / Architecture	Claude-3.7-Sonnet, GPT-o1	Gemini-2.5-Pro	\$0.15-0.60
Generation de Code	Claude-3.7-Sonnet, DeepSeek-R1	GPT-4o	\$0.09-0.45
Generation Config/JSON	Gemini-2.0-Flash	Llama-3.3	\$0.003-0.01
Testing / Validation	Gemini-2.0-Flash, GPT-4o	Claude-3.7-Sonnet	\$0.01-0.19
Optimisation Code	DeepSeek-R1, Claude-3.7-Sonnet	Mistral-Large	\$0.04-0.90
Deployment/DevOps	GPT-4o, Gemini-2.0-Flash	Llama-3.3	\$0.01-0.25

Tableau 2 : Allocation des modeles par type de sous-tache

2.2 Impact sur les Agents Cibles

Pour l'Agent Developpeur, le MoA transforme le pipeline sequentiel en un pipeline parallele : au lieu d'avoir un seul Generateur, on a 2-3 Generateurs travaillant en parallele, dont les resultats sont agreges par l'Agent de Reflexion. Cela augmente le taux de succes de 15-20% (de ~75% a ~92%) au cout d'une multiplication par 2-3 des appels LLM. Le cache semantique et le routing intelligent compensent partiellement ce surcout. Pour l'Agent SLIDES, le MoA est applique differemment : chaque sous-agent (Structure, Contenu, Design) peut appeler 2-3 modeles en parallele pour les taches critiques (generation du outline principal, creation de visuels complexes), et l'Agent Validation utilise le mecanisme de vote multi-criteres pour evaluer la qualite globale.

3. DAG-Based Code Generation : Le coeur du moteur

Le DAG-based Code Generation est le mecanisme qui permet de generer 50-100 fichiers en 2-5 minutes au lieu de 15-30 minutes avec une generation sequentielle. Le principe est de construire un graphe oriente acyclique (DAG) des dependances entre fichiers, puis de generer les fichiers niveau par niveau, avec une generation parallele au sein de chaque niveau. C'est un mecanisme fondamental qui n'etait pas du tout couvert dans le premier document.

3.1 Construction du DAG

Le DAG est construit en deux passes. La premiere passe cree les noeuds pour chaque fichier a generer, chaque noeud contenant le chemin du fichier, son type, son langage et sa description. La deuxieme passe assigne un niveau a chaque noeud en fonction de ses dependances : les fichiers sans dependance (types, schemas, configurations) sont au niveau 0, les fichiers qui dependent du niveau 0 sont au niveau 1, et ainsi de suite. Cette structure hierarchique garantit que chaque fichier est genere avec acces au code de ses dependances deja generees, ce qui elimine les incoherences d'interface.

Niveau	Type de Fichiers	Exemples	Generation
Niveau 0	Configuration, Types, Schemas	tsconfig.json, types/index.ts, schema.sql	Parallele
Niveau 1	Utilitaires, API Client, Lib	lib/api.ts, lib/auth.ts, db/client.ts	Parallele
Niveau 2	Middleware, Routes API	routes/auth.ts, routes/projects.ts	Parallele
Niveau 3	Composants UI, Pages	components/Button.tsx, pages/Dashboard.tsx	Parallele
Niveau 4	Pages composees, App	App.tsx, main.tsx	Sequentiel

Tableau 3 : Niveaux du DAG de generation de code

3.2 Contexte Lineaire (Linear Context Management)

Le mecanisme de Contexte Lineaire est crucial pour la coherence du code genere. Plutot que d'injecter l'integralite du code deja genere dans le prompt de chaque nouveau fichier (ce qui depasserait rapidement la fenetre de contexte du LLM), le systeme injecte uniquement le code des dependances directes du fichier en cours de generation. Pour un composant Button.tsx au niveau 3, le contexte inclut les types du niveau 0 et les utilitaires du niveau 1 qu'il importe, mais pas les autres composants du meme niveau. Cela maintient la taille du contexte sous 8K tokens par fichier, garantit la coherence des interfaces et evite les conflits de noms. C'est un mecanisme essentiel pour la scalabilite de la generation.

Pour l'Agent SLIDES, le meme principe s'applique : chaque slide est generee avec le contexte de l'outline global et des slides adjacentes, mais pas de l'integralite de la presentation. Le Superviseur gere le contexte lineaire en passant uniquement les informations necessaires a chaque sous-agent, ce qui permet de generer des presentations de 50+ slides sans depasser les limites de contexte des LLMs.

4. 3-Level Auto-Fix Engine : Correction automatique

Le moteur de correction automatique a 3 niveaux est le mecanisme qui transforme un taux de succes brut de ~70% (sortie directe du LLM) en ~92% (apres correction). Chaque niveau est plus couteux que le precedent, mais aussi plus precis. Le systeme essaie d'abord le niveau le moins cher et n'escalade que si les erreurs persistent. Ce mecanisme etait insuffisamment detaille dans le premier document qui

mentionnait simplement un "Correcteur".

4.1 Niveau 1 : Corrections par Pattern

Le premier niveau applique des corrections basees sur des patterns regex predefinis. C'est le niveau le plus rapide (milliseconds) et le moins cher (zero appel LLM). Les patterns courants incluent : ajout d'imports manquants (si une variable est utilisee sans import, le pattern detecte le module source et ajoute l'import), correction de points-virgules manquants, normalisation des guillemets, et correction des erreurs de formatage courantes. Ce niveau corrige environ 40% des erreurs sans aucun cout LLM supplementaire.

4.2 Niveau 2 : Corrections par AST

Le deuxieme niveau utilise le compilateur TypeScript comme parseur AST (Abstract Syntax Tree) pour detecter et corriger les erreurs structurelles. Le code genere est compile, les erreurs de type sont collectees, et le systeme applique des corrections automatiques basees sur l'arbre syntaxique : ajout de type assertions, correction des signatures de fonction, resolution des references circulaires, et correction des imports de types vs valeurs. Ce niveau corrige environ 30% des erreurs restantes avec un cout minimal (CPU uniquement, pas de LLM).

4.3 Niveau 3 : Corrections par LLM

Le troisieme niveau est le plus couteux mais le plus puissant. Il envoie le code errone plus la liste des erreurs restantes a un LLM (typiquement Claude-3.7-Sonnet pour sa capacite de raisonnement sur le code) avec un prompt specialise de correction. Le LLM analyse les erreurs dans le contexte du code et genere une version corrgee. Ce niveau corrige les 20-30% d'erreurs restantes, portant le taux de succes global a 92%+. La boucle est limitee a 3 iterations maximum pour eviter les cycles infinis. Si apres 3 iterations le code est encore errone, le fichier est marque comme "needs manual review".

Niveau	Methode	Cout	Taux correction	Latence
Niveau 1	Pattern Regex	0\$ (CPU)	~40%	<100ms
Niveau 2	AST TypeScript	0\$ (CPU)	~30%	<2s
Niveau 3	LLM (Claude/GPT)	\$0.05-0.15/fichier	~25%	5-15s
Cumule	3 niveaux en cascade	\$0.05-0.15/fichier	~92%	5-17s

Tableau 4 : Efficacite du moteur Auto-Fix a 3 niveaux

5. Reflection Multi-Criteres : La cle des 92%

Le premier document mentionnait le pattern Debat (Pro vs Contra + Juge) pour la validation. Le second document revele un mecanisme beaucoup plus sophistique : la Reflection Multi-Criteres avec 6 dimensions ponderees et un score de confiance quantitatif. C'est ce mecanisme qui permet d'atteindre 87.8% sur GAIA et potentiellement 92%+ avec un bon Auto-Fix Engine.

Critere	Poids	Evaluation	Seuil critique
Qualite du Code	30%	Clean, SOLID, DRY, types, pas de TODO	<0.80 = raffiner
Fonctionnalite	25%	Toutes features, edge cases, validation	<0.80 = raffiner
Performance	15%	Algos, memoization, pas de fuites	<0.70 = raffiner
Securite	15%	Validation input, injection, XSS, auth	<0.80 = bloquant
Experience Utilisateur	10%	Responsive, ARIA, loading, erreurs	<0.60 = raffiner
Tests	5%	Coverage, edge cases, integration	<0.50 = raffiner

Tableau 5 : Criteres de Reflection Multi-Criteres avec seuils

Le mecanisme de decision est le suivant : si le score de confiance global est superieur a 0.95, le resultat est accepte directement. Si le score est entre 0.85 et 0.95, un cycle de raffinement est declenche avec les recommandations de l'Agent de Reflection comme contexte supplementaire. Si le score est inferieur a 0.85 ou si un critere critique (securite ou fonctionnalite) est inferieur a 0.80, le systeme regenere le fichier depuis le debut avec un prompt enrichi des erreurs identifiees. Ce mecanisme de seuils est essentiel pour eviter les boucles infinies tout en garantissant un niveau de qualite minimum.

6. Routeur RL-Optimise : Apprentissage du routing

Le premier document decrivait un routeur de modeles statique (classification > routing). Le second document introduit un mecanisme d'apprentissage par renforcement (RL) qui ameliore les decisions de routing au fil du temps. Ce mecanisme est stocke dans la table `rl_training_data` du schema de base de donnees, et utilise les resultats passes (succes/echec, score de qualite, cout, feedback utilisateur) pour calculer une recompense qui guide les futures decisions de routing.

Le signal de recompense combine quatre facteurs : (1) le score de qualite du resultat (0-1), (2) le cout en tokens du modele selectionne, (3) le temps de generation, et (4) le feedback utilisateur (-1 negatif, 0 neutre, 1 positif). La formule de recompense est ponderee pour favoriser la qualite tout en penalisant les couts excessifs : $\text{Reward} = 0.5 * \text{Qualite} - 0.2 * \text{Cout_normalise} - 0.1 * \text{Latence_normalisee} + 0.2 * \text{Feedback}$. Au fur et a mesure que le systeme accumule des donnees d'entrainement, le routeur apprend quel modele est le plus performant pour chaque type de tache et chaque niveau de complexite, ameliorant progressivement le rapport cout/qualite.

En pratique, le routeur RL-optimise est implemente avec une strategie epsilon-greedy : 90% du temps, le routeur selectionne le modele qui a historiquement le meilleur score de recompense pour le type de tache donne ; 10% du temps, il explore un modele alternatif pour collecter de nouvelles donnees. Cette strategie equilibre l'exploitation des connaissances acquises avec l'exploration de nouvelles possibilites, et permet au systeme de s'adapter automatiquement quand de nouveaux modeles sont ajoutes ou quand les performances des modeles existants changent.

7. Architecture Unifiee : Integration complete

Cette section integre les mecanismes des deux documents sources dans une architecture unifiee pour l'Agent Developpeur et l'Agent SLIDES. L'architecture combine les fondations theoriques du Compilateur d'Architecture Intelligente avec les mecanismes operationnels du Guide GenSpark, en ajoutant les specificites de l'Agent SLIDES qui n'etaient pas couvertes.

7.1 Agent Developpeur : Architecture Unifiee

Module	Fonction	Mecanismes Integres	Modele(s)
Analyseur	Decomposition requete	MoA routing + RL optimizer	Sonnet + GPT-4o
Planificateur	Plan + DAG construction	DAG builder + Linear Context	Sonnet + o1
Generateur	Code parallele	MoA 2-3 modeles + DAG levels	Sonnet + DeepSeek-R1
Auto-Fix	Correction 3 niveaux	Pattern > AST > LLM	Flash + Sonnet
Reflection	Validation multi-criteres	6 criteres ponderes + vote	Sonnet (juge)
Correcteur	Raffinement cible	Seuils 0.95/0.85 + feedback RL	Sonnet + DeepSeek
Livreur	Deploy + Git	Sandbox E2B + Cloudflare	Flash

Tableau 6 : Architecture unifiee de l'Agent Developpeur

Le flux unifie est le suivant : l'utilisateur soumet une requete, l'Analyseur la decompose en sous-taches avec le MoA router qui selectionne les modeles optimaux via le RL optimizer. Le Planificateur construit le DAG de dependances entre fichiers. Le Generateur execute les niveaux du DAG en parallele, avec 2-3 modeles par fichier critique via le MoA. L'Auto-Fix applique les 3 niveaux de correction. L'Agent de Reflection evalue les resultats avec les 6 criteres ponderes et un seuil de confiance de 0.95. Si le score est insuffisant, le Correcteur raffine avec les recommandations de la Reflection. Enfin, le Livreur deploye en sandbox E2B pour validation, puis sur Cloudflare Pages pour le deployment production.

7.2 Agent SLIDES : Architecture Unifiee

Sous-Agent	Fonction	Mecanismes Integres	Modele(s)
Superviseur	Coordination + contexte lineaire	Linear Context + MoA orchestration	Sonnet
Structure	Outline + narrative arc	MoA 2 modeles + Reflection seuil	Sonnet + o1
Contenu	Texte + donnees + citations	DAG slides + RAG hybride	GPT-4o + RAG
Design	Layout + visuels + palette	MoA + Image Gen + Template Learning	GPT-4o + DALL-E
Validation	Coherence + qualite + accessibilite	Reflection 6 criteres + Auto-Fix	Sonnet (juge)

Tableau 7 : Architecture unifiee de l'Agent SLIDES

Pour l'Agent SLIDES, le DAG est construit differemment : les slides sont organisees en sections (introduction, points cles, conclusion), et chaque section est un niveau du DAG. Les slides d'introduction (niveau 0) sont generees en premier, fournissant le contexte thematique pour les slides de contenu (niveau 1), qui elles-memes fournissent les donnees pour les slides de synthese et conclusion (niveau 2). Le contexte lineaire garantit que chaque slide est coherente avec les slides precedentes sans necessiter l'integralite de la presentation dans le prompt. Le Template Learning ameliore les presentations futures en memorisant les layouts, palettes et structures narratives qui ont reeu les meilleurs scores de la part des utilisateurs.

8. Infrastructure Critique et Deploiement

8.1 Sandbox E2B / Docker

Le second document detaille l'infrastructure sandbox qui est le coeur de la securite de l'Agent Developpeur. Deux options sont disponibles : E2B Firecracker (micro-VMs isolees, 2-4 vCPU, 4-8 GB RAM) pour la production, et Docker (warm pool de 5 conteneurs pre-chauffes) pour le developpement et les environnements a budget reduit. Le warm pool est un mecanisme critique : au lieu de creer un conteneur a chaque requete (5-10 secondes), les conteneurs sont pre-creees et pre-chauffees avec les dependances courantes (Node.js, Python, Go, Rust, Docker, Git). Quand une requete arrive, un conteneur est immediatement disponible, et un nouveau conteneur est cree en arriere-plan pour recharger le pool. Cela reduit la latence de sandbox de 5-10s a moins de 500ms.

La securite du sandbox repose sur cinq couches : (1) isolation reseau (NetworkMode: none), (2) limites de ressources (CPU 80%, RAM 8GB), (3) timeout d'execution (10 minutes maximum), (4) nettoyage

automatique (30 minutes d'inactivite), et (5) liste noire de commandes dangereuses (rm -rf /, fork bombs, wget|sh, etc.). Le sandbox utilise un utilisateur non-root (uid 1000) et un systeme de fichiers temporaire pour /tmp avec les options noexec et nosuid.

8.2 Stack de Deploiement

Composant	Production	Alternative (Budget)
Frontend Hosting	Cloudflare Pages (edge, SSL auto)	Vercel / Netlify
API Runtime	Cloudflare Workers (edge, Hono)	Railway / VPS Docker
Base de donnees	Cloudflare D1 (SQLite edge)	Supabase PostgreSQL
Cache/Queue	Upstash Redis (serverless)	Redis auto-heberge
Stockage fichiers	Cloudflare R2 (S3-compatible)	S3 / MinIO
Sandbox execution	E2B Firecracker (isole)	Docker warm pool
CI/CD	GitHub Actions + ArgoCD	GitHub Actions simple
Observabilite	OTEL + Grafana + ClickHouse	Sentry + logs basiques
Cout mensuel estime	\$42-775	\$6-100

Tableau 8 : Stack de deploiement production vs budget

8.3 Schema de Base de Donnees Critique

Le second document fournit un schema de base de donnees complet qui inclut des tables specifiques aux mecanismes avances. La table rl_training_data stocke les donnees d'apprentissage par renforcement avec le contexte de la tache, l'action du routeur (modele selectionne), le resultat (succes, score de qualite, temps, cout) et la recompense calculee. La table error_recovery_log trace chaque tentative de correction avec le niveau utilise (pattern, AST, LLM), le succes ou l'echec, et le code avant/apres. La table cost_tracking agrege les couts par utilisateur et par jour, avec le detail par modele. La table analytics_events capture tous les evenements produit pour le dashboard de metriques. Ces tables sont essentielles pour le RL optimizer, l'Auto-Fix Engine et le systeme de credits du document original.

9. Optimisation des Couts : Modele economique

Le second document fournit des estimations de couts detaillees qui sont directement actionnables. Les couts LLM dominent largement les couts d'infrastructure (ratio 18:1), ce qui confirme la Theorie des Contraintes du premier document : le LLM est le goulot d'etranglement. L'optimisation des couts passe donc principalement par le routing intelligent des modeles.

Type de Projet	Modeles Utilises	Tokens Total	Cout LLM	Temps
Simple (landing)	Flash + Sonnet	~13K	\$0.09	30-60s
Full-stack (React+Hono)	Sonnet + GPT-4o + DeepSeek + Flash	~62K	\$0.67	2-3min
Production (SaaS complet)	Sonnet + o1 + GPT-4o + DeepSeek + Gemini	~103K	\$1.85	3-5min

Tableau 9 : Estimation des couts LLM par type de projet

Les leviers d'optimisation des couts, combines des deux documents, sont : (1) le cache semantique (40-60% de reduction des appels LLM), (2) le routing RL-optimize (selection du modele le moins cher qui repond au besoin de qualite), (3) le prompt caching (50-90% de reduction pour les prompts systemes reutilisables), (4) la generation DAG parallele (reduit le temps total mais pas le cout direct, sauf si les modeles moins cher sont utilises pour les niveaux simples), et (5) l'Auto-Fix Niveau 1 et 2 qui corrigent 70% des erreurs sans appel LLM supplementaire. Combine, ces leviers peuvent reduire le cout moyen par projet de 40-60%.

10. Roadmap Unifiee : 16 Semaines

Phase	Semaines	Livrables	Mecanismes
Fondation	1-3	MVP Agent Dev (pipeline basique), API, Auth, DB	Pipeline simple + Auto-Fix N1
Intelligence	4-6	RAG hybride, MoA routing, Cache semantique	MoA 3 modeles + Reflection
Generation	7-9	DAG builder, Auto-Fix 3 niveaux, Sandbox	DAG + Linear Context + 3-Level Fix
Apprentissage	10-11	RL Router, Template Learning, Memoire 5 couches	RL optimizer + Auto-amelioration
SLIDES	12-13	Agent SLIDES (Supervisor + 4 sous-agents)	MoA + DAG slides + Design Gen
Production	14-16	Securite 5 couches, Observabilite, Scaling	Deploy auto + Cost optimizer

Tableau 10 : Roadmap unifiee sur 16 semaines

Cette roadmap unifiee integre les mecanismes des deux documents en 16 semaines. Les semaines 1-3 posent les fondations avec un pipeline simple et l'Auto-Fix Niveau 1 (pattern-based). Les semaines 4-6 ajoutent le MoA avec 3 modeles minimum et l'Agent de Reflection avec seuils de confiance. Les semaines 7-9 implementent le DAG builder, le contexte lineaire et l'Auto-Fix a 3 niveaux avec le

sandbox E2B/Docker. Les semaines 10-11 deployent le RL optimizer et le Template Learning pour l'amelioration continue. Les semaines 12-13 construisent l'Agent SLIDES avec le pattern Supervisor et les mecanismes MoA/DAG adaptes aux presentations. Les semaines 14-16 finalisent avec la securite en profondeur, l'observabilite complete et l'auto-scaling.

11. Les 10 Nouveaux Commandements (Addendum)

En complement des 15 commandements du premier document, voici les 10 nouveaux commandements issus de l'analyse du second document et de l'integration des mecanismes avances.

16. Mixture-of-Agents obligatoire pour les taches critiques. Un seul modele ne suffit jamais pour atteindre 90%+ de taux de succes.
17. DAG-based generation pour tout projet > 10 fichiers. La generation sequentielle est un anti-pattern de performance.
18. Auto-Fix a 3 niveaux avant toute intervention humaine. 70% des erreurs sont corrigeables sans LLM supplementaire.
19. Reflexion avec seuils quantitatifs, pas d'evaluation subjective. Score < 0.95 = raffiner, < 0.85 = regenerer.
20. Contexte lineaire pour respecter les fenetres de tokens. Injecter uniquement les dependances directes, jamais tout le projet.
21. RL optimizer des le jour 1, pas en phase d'optimisation. Chaque generation produit des donnees d'entrainement.
22. Warm pool de sandboxes pour reduire la latence a < 500ms. Les 5-10 secondes de creation Docker sont inacceptables en production.
23. Cout LLM > Cout infra dans 95% des cas. Optimisez le routing avant d'optimisez les serveurs.
24. Template Learning pour amelioration continue. Les patterns reussis doivent etre memorises et reutilises automatiquement.
25. Seuil de securite 0.80 non negociable. Si le score securite < 0.80, bloquer meme si le score global > 0.95.